Quickstarts

Quickstart: Bridge between Solana and EVM

Copy page

Use App Kit to bridge USDC between Solana and an EVM-compatible blockchain

This quickstart helps you write a server-side script that bridges USDC between Solana and an EVM-compatible blockchain. The examples in this quickstart use Solana Devnet and Arc Testnet, but you can use Solana and any **supported EVM chain** as source or destination.

Prerequisites

Before you begin, ensure that you've:

- Installed **Node.js v22+**.
- Created a Solana Devnet wallet and an **Arc Testnet** wallet using a wallet provider such as **MetaMask**.
- Funded your wallets with testnet tokens:
  - Get testnet USDC from the **Circle Faucet**.
  - Get native tokens for Solana Devnet from the **Solana Faucet**.
  - Get native tokens for Arc Testnet from the **Console Faucet**.

Step 1. Set up the project

This step shows you how to prepare your project and environment.

1.1. Set up your development environment

Create a new directory and install App Kit and its dependencies:

Shell

```shell
# Set up your directory and initialize the project
mkdir app-kit-quickstart-bridge-solana-evm
cd app-kit-quickstart-bridge-solana-evm
npm init -y

# Install App Kit and tools (Solana + EVM adapters)
npm install @circle-fin/app-kit @circle-fin/adapter-viem-v2 @circle-fin/adapte
```

Only need to bridge and want a lighter install than the full App Kit? Install the standalone bridge package instead: `@circle-fin/bridge-kit`

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

Shell

```shell
npx tsc --init
```

Then, update the `tsconfig.json` file:

Shell

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Configure environment variables

Create an `.env` file in the project directory:

Shell

```
touch .env
```

Then, add your wallet private keys. Replace `YOUR_SOLANA_PRIVATE_KEY` with your Solana wallet private key and `YOUR_ARC_TESTNET_PRIVATE_KEY` with your Arc Testnet (EVM) wallet private key:

.env

```
SOLANA_PRIVATE_KEY=YOUR_SOLANA_PRIVATE_KEY
EVM_PRIVATE_KEY=YOUR_ARC_TESTNET_PRIVATE_KEY
```

If you use MetaMask, follow their guide for how to **find and export your private key**.

Edit `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Bridge USDC

This step shows you how to set up your script, execute the bridge transfer, and check the result.

2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code sets up your script and bridges 1.00 USDC from Solana Devnet to Arc Testnet:

> Using a different EVM chain or Solana as the destination? Change the `chain` values in `kit.bridge()` and ensure your source wallet has USDC and both wallets have native tokens.

TypeScript

```typescript
// Import App Kit and dependencies
import { AppKit } from "@circle-fin/app-kit";
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { createSolanaKitAdapterFromPrivateKey } from "@circle-fin/adapter-sola
import { inspect } from "util";

const kit = new AppKit();

const bridgeUSDC = async (): Promise<void> => {
  try {
    const solanaAdapter = createSolanaKitAdapterFromPrivateKey({
      privateKey: process.env.SOLANA_PRIVATE_KEY as string,
    });
    const evmAdapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.EVM_PRIVATE_KEY as string,
    });

    console.log("---------------Starting Bridging---------------");

    const result = await kit.bridge({
      from: { adapter: solanaAdapter, chain: "Solana_Devnet" },
      to: { adapter: evmAdapter, chain: "Arc_Testnet" },
      amount: "1.00",
    });

    console.log("RESULT", inspect(result, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void bridgeUSDC();
```

You can customize your bridges to **collect a fee**, **use the CCTP Forwarding Service**, or **estimate gas and provider fees** before bridging. Proceed only if the cost works for you.

2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

Shell

```
npx tsx --env-file=.env index.ts
```

2.3. Verify the transaction

After the script finishes, find the returned `steps` array in the terminal output. Each transaction step includes an `explorerUrl`. Use that link to verify that the USDC amount matches the amount you bridged.

The following code is an example of how an `approve` step might look in the terminal output. The values are used in this example only and are not a real transaction:

Shell

```
steps: [
  {
    name: "approve",
    state: "success",
    txHash: "0xdeadbeefcafebabe1234567890abcdef1234567890abcdef1234567890abcd"
    data: {
      txHash:
        "0xdeadbeefcafebabe1234567890abcdef1234567890abcdef1234567890abcd",
      status: "success",
      cumulativeGasUsed: 17138643n,
      gasUsed: 38617n,
      blockNumber: 8778959n,
      blockHash:
        "0xbeadfacefeed1234567890abcdef1234567890abcdef1234567890abcdef12",
      transactionIndex: 173,
      effectiveGasPrice: 1037232n,
      explorerUrl:
        "https://testnet.arcscan.app/tx/0xdeadbeefcafebabe1234567890abcdef1234
    },
  },
];
```

Was this page helpful? Yes No

Bridge between EVM chains

Bridge with Circle Wallets

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.